

Where Did the Agent Go Wrong? Evaluating Failure Attribution Against Counterfactual Rescue

Anonymous ACL submission

Abstract

Self-improving agent loops start by attributing each failure to a step of the episode, usually by asking an LLM to read the trace. We ask how well attribution methods find the step whose correction would have rescued a failed episode, at what cost, and whether better attribution gives better repairs. Defining ground truth as the earliest such step proved unstable: a failure often has several rescuable steps, and two independent runs named the same step in 42% of comparisons. We instead measure, for every step, how much a single correction raises the success of continued replays above replays without it, and score a method by this rescue gain at the step it names. On 300 natural failures from a running self-improvement loop in three domains, τ^2 -bench retail and airline and AppWorld with a code agent, with registered analyses, no LLM-based method did better than a zero-cost rule naming the first state-changing action (mean rescue gain 0.42, against 0.20–0.34). Counterfactual search with up to 40 replays was no better than one judge call (τ^2 : -0.06 [-0.15 , $+0.03$]; AppWorld: $+0.03$ [-0.09 , $+0.14$]) at seven times the cost, and a small model’s judge matched a large model with thinking at a fourteenth of the cost. The best simple rule changed with the domain: in AppWorld, where decisive steps came late, naming the last step did as well as any judge. Binary search was the only method whose accuracy held across domains. In a second registered experiment, we gave each attribution to the loop’s proposer at eight states of the loop in τ^2 and deployed the 54 resulting repairs on held-out tasks: repairs drafted from the first-write rule or binary search were practically equivalent to those drafted from the loop’s own, less accurate analysis ($+0.3$ points of held-out success, 90% intervals within ± 2.5), and counterfactual search did not help. Attribution benchmarks need rescue-based ground truth with null controls, domain baselines and a test on repairs before LLM attributors can be

said to help.

1 Introduction

Agents that improve themselves from their own failures share a first step. An analyst model reads a failed episode, decides where it went wrong, and hands that diagnosis to a proposer that edits the agent’s prompts, tools or code (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2026b; Chen et al., 2026; Xia et al., 2026). If the diagnosis points at the wrong step, the proposer repairs the wrong thing. Failure attribution has therefore become a task of its own, with benchmarks of human-labelled multi-agent logs (Zhang et al., 2025; Liu et al., 2026), taxonomies of failure modes (Cemri et al., 2025, 2026; Zhu et al., 2025), and methods that replay a failure with a step changed (Zhang et al., 2026a; Ma et al., 2025; Lin et al., 2026; Bonagiri et al., 2026; Shah, 2026).

Two questions are rarely asked together: is an attribution correct in the sense a repair loop needs, and is it worth what it costs? A human label says which step an annotator blames; a repair loop needs the step whose correction would have made the episode succeed. Replay-based methods test that directly but spend agent episodes; an LLM judge spends one call. We evaluate both kinds against a ground truth defined by rescue and report cost next to accuracy.

Defining that ground truth turned out to be the hard part. Our registered plan named the *earliest* step whose single correction beats replays without correction. On a pilot, two independent runs of this protocol agreed on 42% of failures (§3): a failure usually has several steps whose correction rescues it, and which one is found first depends on which corrections an oracle happens to write. We therefore measure a *rescue profile*: for every step, the mean gain in success over null replays across four independent corrections. A method is scored by the rescue gain at the step it names, and

a failure is *rescuable* when some step reaches a gain of 0.5. Null replays matter: continuing an unchanged failed episode from a step succeeded 11–27% of the time, depending on the domain.

We apply the profile to 300 natural failures of agents built by a self-improvement loop, RRSI (Xia et al., 2026), in τ^2 -bench’s retail and airline domains (Barres et al., 2025) and in AppWorld with a code agent (Trivedi et al., 2024), and compare twelve attributions: three all-at-once judges of different cost, step-by-step and binary-search judges (Zhang et al., 2025), a root-cause attributor, the trace digester RRSI uses, counterfactual search with up to 40 replays, and two structural rules. Methods, metrics and analyses were registered before any ground truth was computed. A second registered experiment re-runs one round of RRSI at eight of its own states in τ^2 with each attribution, or none, given to the proposer, and deploys every resulting repair on 240 held-out episodes (§6). Our findings:

- No LLM-based method did better than naming the first state-changing action, a rule that costs nothing (§5). The best LLM method, binary search, was 0.08 lower (exploratory interval $[-0.02, +0.18]$).
- The registered comparison of counterfactual search against a single judge call found no difference in either τ^2 or AppWorld, and search cost seven times as much. The cheapest judge matched the most expensive one.
- Which simple rule wins depends on the domain. In retail most decisive steps are writes; in airline most are utterances, and the judges led; in AppWorld decisive steps come late and are often the submission, so naming the last step scored 0.36, against 0.11–0.13 in τ^2 .
- More accurate attribution did not give better repairs. Repairs drafted from first-write or binary-search steps were practically equivalent to those drafted from RRSI’s own analysis, which had located decisive steps less well (+0.31 and +0.29 points of held-out success; 90% intervals within ± 2.5 points), and those drafted from counterfactual search were not better (§6). The largest losses came from single harmful edits, which RRSI’s own re-evaluation rejected.
- On Who&When, our judges reproduce the published gap between agent-level and step-level accuracy (§5).

2 Related Work

Failure attribution. Who&When introduced automated attribution of the responsible agent and step in multi-agent logs, with all-at-once, step-by-step and binary-search judges, and found step-level accuracy far below agent-level accuracy (Zhang et al., 2025); Who&When Pro revisits these labels (Liu et al., 2026). Taxonomies organise failures into modes, including adaptive ones (Cemri et al., 2025, 2026), and AgentDebug pairs a taxonomy with targeted feedback (Zhu et al., 2025, 2026); ErrorProbe improves diagnosis from its own history (Li et al., 2026). These evaluations score agreement with human or injected labels; we score the effect of correcting the named step.

Counterfactual attribution. AgenTracer labels failures by replaying corrected steps of injected faults and trains an attributor on them (Zhang et al., 2026a). DoVer and REFLECT validate hypotheses by intervening on a trace (Ma et al., 2025; Lin et al., 2026), CausalFlow and Causal Agent Replay attribute failures by counterfactual replay (Bonagiri et al., 2026; Shah, 2026). Our ground truth uses the same intervention, with two differences that turned out to matter: null replays from the same step, without which a fifth to a quarter of τ^2 replays would count as rescues, and a full profile over steps instead of the first step that flips. The profile is a graded, sampled version of but-for causation (Halpern, 2016; Pearl, 2009).

Self-improving agents. Reflexion, ExpEL and ACE turn failures into lessons or playbooks (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2026b); HarnessFix diagnoses and repairs harness flaws (Chen et al., 2026); RRSI digests failed traces, proposes harness edits and keeps them by re-evaluation (Xia et al., 2026). Prediction of failure does not imply that intervening helps (Vasudev et al., 2026). We evaluate the diagnostic step these loops share, on failures one of them produced.

3 Ground Truth: Rescue Profiles

Replays. A failed episode is a sequence of agent steps $0, \dots, n-1$. A *replay from step k* keeps steps $0, \dots, k-1$ and the environment state they produced, and lets the same agent and user simulator

continue; a *corrected* replay forces a given action at step k first. The agent runs at temperature 0, but replays still vary, so each outcome is a sample.

Oracle corrections. For each step an oracle (DeepSeek-V4-Pro, thinking off) sees the domain policy, the tools, the episode up to the step and the task’s grading criteria: τ^2 ’s gold actions and required information, or AppWorld’s unit tests with their expected values. It either judges the step correct or writes the action the agent should have taken, using only what the agent knew at that point: the criteria tell it the right outcome, not identifiers or facts the agent had not yet seen. The LLM attributors we evaluate read the same grading report, as RRSI’s analyst does (§4); only counterfactual search works without it.

From earliest step to profile. Our first registered protocol named the earliest step whose correction succeeded on at least 2 of 4 replays more than 4 null replays did. On 20 pilot failures, four independent runs of it named the same decisive step in 42% of pairs of runs (50 of 120), and in 23% of pairs where either run found one. Most disagreements came from the oracle, not from replay noise: a step judged correct in one run and corrected in another, or a correction that worked in one run and a different correction that did not in the next. Before computing any ground truth on the main set, we registered protocol 3 instead.

Protocol 3. At every step k we draw $K = 4$ oracle samples. Each sample that corrects the step gets 2 corrected replays, and the step gets 4 null replays. The rescue gain of step k is

$$R_k = \frac{1}{K} \sum_{i=1}^K (\hat{p}_{k,i}^{\text{corr}} - \hat{p}_k^{\text{null}}), \quad (1)$$

where a sample judging the step correct, or giving an action the replay cannot apply, contributes 0. A failure is rescuable if $\max_k R_k \geq 0.5$, and then its decisive step is the step with the largest gain (earliest on ties). We also record the earliest step with $R_k \geq 0.5$. On the pilot, step-level gains of two independent runs correlated .74 (retail) and .64 (airline), and the decisive step agreed on 15 of 20 failures; where both runs found one, on 7 of 8, the eighth one step apart.

Scoring a method. A method names one step $\hat{k}$. Its score on a failure is $R(\hat{k})$, the rescue gain at that step, and 0 for an invalid step. The primary

metric is the mean of $R(\hat{k})$ over rescuable failures; it rewards naming any step whose correction helps, in proportion to how much it helps. Secondary metrics are exact agreement with the decisive step, agreement within one step, agreement with the earliest rescuing step, the share $R(\hat{k}) / \max_k R_k$, and $R(\hat{k})$ over all failures.

4 Setup

Failures. All failures come from RRSI (Xia et al., 2026) run as a self-improvement loop on a DeepSeek-V4-Flash agent: from the harnesses it accepted over 20 rounds, in their evaluations on training tasks and in their deployment on held-out tasks. In τ^2 -bench (Barres et al., 2025) the agent serves a simulated customer through tool calls; in AppWorld (Trivedi et al., 2024) it writes Python against the APIs of nine apps and submits with a final call. We drew 100 failed episodes per domain with a registered rule: no harness error, at most two per task and harness, tasks taken round-robin in a seeded order. They cover 36 (retail), 24 (airline) and 47 (AppWorld) tasks and average 11.7, 10.3 and 19.6 agent steps.

Methods. Every LLM method except counterfactual search reads the failure as RRSI’s analyst reads it: the task (in τ^2 with the simulated customer’s hidden instructions), the episode with numbered steps, and the grading report the agent never sees, which lists τ^2 ’s gold actions and missing information or AppWorld’s failed unit tests with their assertions. The judges therefore have the information the ground-truth oracle has. Each method names one step.

- **Rules:** the *first write*, the first step that calls a state-changing tool or API (else the last step), and the *last step*.
- **All-at-once** judge (Zhang et al., 2025): one call over the whole episode, with DeepSeek-V4-Flash, V4-Pro, and V4-Pro with thinking.
- **Step-by-step** judge: one call per step, in order, until a step is judged decisive.
- **Binary search** judge: halves the episode $\lceil \log_2 n \rceil$ times, asking which half holds the decisive mistake.
- **Root-cause attributor:** one call asking for a root cause, the earliest wrong step and a component.

- **RRSI trace digester** (Xia et al., 2026): RRSI’s own multi-turn failure digest; we take the earliest step it cites as evidence.
- **Counterfactual search**, replay-based like AgenTracer’s labelling (Zhang et al., 2026a) but without grading criteria: one call names up to five suspect steps with corrected actions; suspects are tested in order with 4 corrected and 4 null replays, and the search stops at the first correction that beats null by 2 successes or when its budget of 40 replays runs out; otherwise it answers with the first suspect. Answers at budgets of 8 to 40 come from the same run.

Judges are prompted for the decisive mistake, defined as the earliest step whose correction would most likely have rescued the episode (Appendix B). Pro runs with thinking off unless stated.

Registration and analysis. Methods, prompts, metrics and analyses were registered, and all methods had been run, before the main ground truth was computed; no method sees ground truth. The registered primary comparison is counterfactual search at budget 40 against the all-at-once judge with Pro: the paired difference in $R(\hat{k})$ over rescuable failures, with a 95% bootstrap interval over task clusters (10,000 draws) and a paired permutation test flipping signs by task, on retail and airline pooled and, for AppWorld, separately. Everything else is estimated with cluster-bootstrap intervals and no further tests; comparisons we did not register are marked exploratory. A seeded 10% of failures in each domain got a second, independent ground truth. Costs are list-price dollars for tokens and replays. Appendix D lists every change made after registration.

5 Results

Ground truth. Of 300 failures, 157 were rescuable: 65 of 100 in retail, 37 in airline and 55 in AppWorld. Null replays from the same step succeeded 27%, 18% and 11% of the time; without them, many rescues would be chance recoveries. Of the rescuable failures, 18, 11 and 26 had two or more steps with a gain of at least 0.5, so a single "correct step" would have been an arbitrary choice for about a third of them. The retest agreed on the decisive step for 25 of 30 failures (9, 8 and 8 of 10), counting failures both runs found unrescuable as

agreeing; the AppWorld disagreements were one and two steps apart.

No method beat the first-write rule. Table 1 gives every method’s rescue gain. Pooled over the three domains, the first write scored 0.42 [0.33, 0.51]; the LLM methods scored between 0.20 (step-by-step) and 0.34 (binary search). The paired difference between the first write and binary search was +0.08 (exploratory, $[-0.02, +0.18]$), and between the first write and the all-at-once judges +0.11 to +0.12. Exact agreement with the decisive step ordered the methods the same way (Appendix Table 4).

Replays did not help. Counterfactual search at budget 40 scored 0.26 pooled. The registered comparison with the all-at-once judge (Pro) found no difference: $-0.061 [-0.146, +0.032]$ in τ^2 (permutation $p = .20$, $n = 102$) and $+0.025 [-0.087, +0.136]$ in AppWorld ($p = .69$, $n = 55$). Search cost \$0.022 per failure against \$0.003 for the judge. It never used its whole budget: after 17 replays on average it had either confirmed a suspect (42% of failures) or tested all of its four or five suspects, so its accuracy stopped rising at 24 replays (Appendix Table 5). Its suspects were not the problem: on 122 of 157 rescuable failures (78%) they included a step with a gain of at least 0.5. Its confirmations were. Of the 87 rescuable failures where its own test (4 corrected against 4 null replays, without grading criteria) confirmed a suspect, the confirmed step had a gain of at least 0.5 in 41; elsewhere its corrections at the right step failed, or a weaker step passed the test first.

Cost bought little. Among the all-at-once judges, Flash (\$0.001 per failure), Pro (\$0.003) and Pro with thinking (\$0.014) scored 0.30, 0.30 and 0.31 pooled (Figure 1). Step-by-step judging, at three times the cost of an all-at-once call with Pro, was worse in τ^2 . RRSI’s own digester, which cites evidence steps for a proposer, scored 0.22.

Judges point too early. When a judge missed the decisive step, it usually named an earlier one. In retail, the all-at-once judge with Pro named an earlier step on 40 of 65 rescuable failures and a later one on 7; the step-by-step judge, which stops at the first step it calls decisive, named an earlier one on 49. The first-write rule erred in both directions (11 earlier, 17 later). The judges were asked for the earliest decisive mistake, but this does not explain the pattern: the earliest step with a gain

Method	Model	Mean rescue gain $R(\hat{k})$, rescuable failures [95%]				Exact (pooled)	\$ per failure
		Retail (65)	Airline (37)	AppWorld (55)	Pooled (157)		
First write	rule	0.53 [0.38, 0.65]	0.27 [0.13, 0.45]	0.39 [0.26, 0.52]	0.42 [0.33, 0.51]	0.38	0.0000
Last step	rule	0.11 [0.02, 0.24]	0.13 [0.04, 0.23]	0.36 [0.23, 0.50]	0.20 [0.13, 0.29]	0.13	0.0000
All-at-once	Flash	0.29 [0.19, 0.38]	0.41 [0.28, 0.53]	0.25 [0.14, 0.37]	0.30 [0.24, 0.37]	0.28	0.0010
All-at-once	Pro	0.30 [0.19, 0.41]	0.34 [0.21, 0.48]	0.26 [0.15, 0.38]	0.30 [0.23, 0.36]	0.31	0.0031
All-at-once	Pro, thinking	0.26 [0.17, 0.35]	0.42 [0.28, 0.57]	0.28 [0.18, 0.40]	0.31 [0.24, 0.38]	0.32	0.0141
Step-by-step	Pro	0.14 [0.03, 0.25]	0.16 [0.04, 0.25]	0.29 [0.18, 0.40]	0.20 [0.13, 0.26]	0.19	0.0099
Binary search	Pro	0.33 [0.21, 0.47]	0.34 [0.21, 0.46]	0.35 [0.24, 0.45]	0.34 [0.27, 0.41]	0.32	0.0064
Root-cause attributor	Pro	0.26 [0.16, 0.38]	0.36 [0.21, 0.53]	0.22 [0.10, 0.34]	0.27 [0.20, 0.34]	0.28	0.0031
RRSI trace digester	Pro	0.17 [0.07, 0.29]	0.35 [0.24, 0.49]	0.19 [0.11, 0.27]	0.22 [0.16, 0.29]	0.21	0.0077
Counterfactual search@40	Pro, replays	0.22 [0.12, 0.33]	0.31 [0.19, 0.44]	0.28 [0.17, 0.41]	0.26 [0.20, 0.34]	0.26	0.0222

Table 1: Attribution scored by the rescue gain at the named step. Intervals: cluster bootstrap over tasks. Exact: share naming the decisive step. Cost: mean over domains of list-price dollars per failure (tokens and replays).

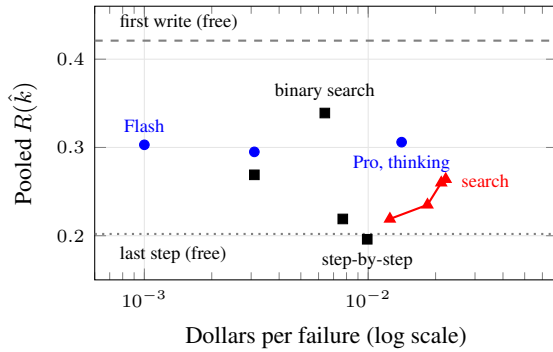


Figure 1: Accuracy against cost. Circles: all-at-once judges; squares: other LLM methods; triangles: counterfactual search at budgets 8, 16, 24 and 40. The free first-write rule lies above every method.

of at least 0.5 was the decisive step in 130 of 157 rescuable failures, and the all-at-once and binary-search judges agreed with it about as often as with the decisive step (0.31–0.36 against 0.28–0.32; Appendix Table 4).

The best rule depends on the domain. The first write led in retail (0.53, against 0.14–0.33 for LLM methods) and in AppWorld (0.39), but in airline the all-at-once judges led (0.41–0.42, against 0.27). Table 2 shows why. In retail, 36 of 65 decisive steps were writes, and the correction replaced a wrong write with the right one; the first write is often that write. In airline, 25 of 37 were utterances, typically a refusal or a clarifying question the agent should have made before acting. In AppWorld, 29 of 55 were writes but only 16 were the first one, and 9 were the final submission, made with a wrong answer or before the work was done; decisive steps lay on average 77% of the way through the episode, so naming the last step scored 0.36,

τ^2	steps/dec.	utterance	read	write	transfer
Retail	1165/65	54%/28	34%/1	11%/36	1%/0
Airline	1026/37	55%/25	31%/4	12%/8	1%/0
AppWorld	steps/dec.	docs	read	write	submit
AppWorld	1961/55	27%/3	45%/9	14%/29	5%/9

Table 2: Share of all agent steps of each kind / number of decisive steps of that kind. AppWorld "docs" looks up API documentation; 5 AppWorld decisive steps (9% of steps) are other code.

against 0.11–0.13 in τ^2 . Binary search was the only LLM method whose accuracy held across domains (0.33, 0.34, 0.35).

Who&When. To check that our judges are not unusually weak, we ran the all-at-once, step-by-step and binary-search judges on the 184 logs of Who&When with its authors’ prompts and the setting that shows the ground-truth answer (Zhang et al., 2025). They identified the responsible agent in 58–67% of logs with all-at-once judging and the responsible step in 3–25% (Appendix Table 6), the same pattern the benchmark reports: agents are found far more often than steps.

6 Do Better Attributions Give Better Repairs?

Design. Accuracy against rescue profiles is a proxy: a loop needs attributions only to repair its agent. We tested this on RRSI itself, in a separately registered experiment. We took two other runs of the loop in each τ^2 domain and the harness each run had accepted by the start of rounds 5 and 15, giving eight incumbent states. At each state we re-ran one unmodified RRSI round five times,

changing only the evidence the proposer gets:

- **No attribution:** a note asking the proposer to read the traces itself.
- **RRSI analysis:** the round’s own analysis report and trace digests, unchanged.
- **First write, binary search (Pro) and counterfactual search@40:** for each failing trace, the step the method named and an excerpt of it, with no reason, component or method name (Appendix B).

The proposer works from the traces RRSI builds from the incumbent’s evaluation on training tasks (81 failing traces in all, 42 retail and 39 airline) and, as in RRSI, can read each of them, grading report included. Each round drafts two candidates and puts them through RRSI’s critic, smoke test and full evaluation on training tasks, after which RRSI’s own rule decides whether to accept one. A candidate is *deployable* if it passes these gates with a harness error rate of at most 2%. Every deployable candidate and every incumbent was deployed on 240 held-out episodes (retail 40 tasks $\times$ 6, airline 20 $\times$ 12).

Analysis. A candidate slot’s gain G is its held-out success minus its incumbent’s, in points, and 0 if it was not deployable, since the loop then keeps the incumbent. A contrast between two groups is the mean over the eight states of the difference in their mean gains. The six primary contrasts compare each step attribution with no attribution and with RRSI’s analysis; p values come from permuting group labels within states and are Holm-adjusted, and 95% intervals from a t distribution over states (df 7). A contrast whose 90% interval lies within ± 2.5 points, about one standard deviation of a single candidate’s gain in earlier runs, counts as practically equivalent. The design, with an expected standard error of 0.9 points per contrast, was registered before any attribution was run on these traces, and no held-out result was looked at until all 62 deployments had finished.

No contrast was significant. Of 80 drafts, 54 were deployable, 9 to 13 per group (Table 3); the others made no proposal, were rejected by the critic or failed the smoke test, and none failed the error check. No primary contrast was significant (Holm-adjusted $p = 1.0$ for all six). Repairs drafted from first-write and binary-search steps were practically equivalent to those drafted from

RRSI’s analysis: $+0.31 [-0.56, +1.18]$ and $+0.29 [-0.82, +1.39]$ points. Counterfactual search was 2.14 points below RRSI’s analysis $[-5.67, +1.40]$, mostly through two airline candidates that lost 23 and 14 points: a gate that blocked writes it judged unconfirmed or inconsistent with the conversation, and an edit that added totals to flight searches and zeroed paid bags in baggage updates. The contrasts with no attribution span about ± 10 points because of a single candidate drafted without attribution. It held back every write whose order identifier had not appeared in an earlier order lookup; its check did not match how lookups were recorded, so it blocked nearly every write without raising an error and lost 72.5 points. Without it, the no-attribution group’s mean gain was $+0.17$ points, in line with the others (exploratory).

Accuracy did not carry over. In retail, the first write had located decisive steps three times as well as RRSI’s digester (0.53 against 0.17), yet repairs drafted from the two differed by $+0.52 [-0.49, +1.54]$ points. The three step methods rarely named the same step (all three agreed on 10% of retail traces and on none in airline), so the proposers did receive different evidence. Most repairs did not help on held-out tasks in any group: 25–60% of deployable candidates gained, and no group’s mean gain was above zero. RRSI’s own rule rejected all three candidates that lost more than 10 points, so at the level of the loop the gain of the accepted candidate differed little between groups: between -0.10 and $+0.47$ points, with no contrast significant (exact sign-flip $p \geq .25$). As checks, the RRSI-analysis group’s mean gain (-0.34) matched that of the candidates the original rounds had drafted from the same evidence (-0.42), and re-deploying each incumbent moved its held-out success by -2.5 to $+3.3$ points against its earlier deployment, within the noise of 240 episodes.

7 Discussion

Why a rule wins. The rescue profile asks where a single correction helps most. In customer service, an agent that changes the database wrongly fails at that write, and correcting it is the most direct rescue; the first write is often that step. The judges, reading the same gold actions, tended to blame earlier steps: imperfect questions or confirmations that a correction did not turn into success. Where the decisive mistake is a conversational act, as in

Evidence for the proposer	$R(\hat{k})$ (Tab. 1)		Mean G , all 16 slots			Deployable		RRSI	Round
	retail / airline	Deployable	both	retail	airline	mean G	$G > 0$	accepted	level
No attribution	–	10/16	−4.38	−9.27	+0.52	−7.00	50%	2	−0.10
RRSI analysis	0.17 / 0.35	13/16	−0.34	−0.31	−0.36	−0.42	38%	1	−0.10
First write	0.53 / 0.27	10/16	−0.03	+0.21	−0.26	−0.04	60%	2	+0.47
Binary search	0.33 / 0.34	9/16	−0.05	+0.16	−0.26	−0.09	44%	0	0.00
Counterfactual search@40	0.22 / 0.31	12/16	−2.47	−0.26	−4.69	−3.30	25%	2	+0.16

Primary contrast	Difference [95%]	p	Holm	Equivalent	Round level [95%]
First write – none	+4.35 [−5.97, +14.67]	0.52	1.00	no	+0.57 [−0.19, +1.34]
Binary search – none	+4.32 [−6.90, +15.55]	0.66	1.00	no	+0.10 [−0.14, +0.35]
CF search@40 – none	+1.90 [−10.24, +14.04]	0.91	1.00	no	+0.26 [−0.99, +1.51]
First write – RRSI	+0.31 [−0.56, +1.18]	0.65	1.00	yes	+0.57 [−0.32, +1.46]
Binary search – RRSI	+0.29 [−0.82, +1.39]	0.60	1.00	yes	+0.10 [−0.14, +0.35]
CF search@40 – RRSI	−2.14 [−5.67, +1.40]	0.31	1.00	no	+0.26 [−0.99, +1.51]

Table 3: Repairs drafted from each kind of evidence in τ^2 : 8 states $\times$ 2 candidates per group. G : held-out success of a candidate minus that of its incumbent, in points, 0 if the candidate was not deployable. $R(\hat{k})$: the method’s rescue gain on rescuable failures in Table 1 (for RRSI’s analysis, its digester). Accepted: candidates RRSI’s own rule accepted. Round level: gain of the accepted candidate, 0 if none. Contrasts: mean over states of group differences, p from within-state permutation; Equivalent: 90% interval within ± 2.5 points. Round-level contrasts are tested by exact sign flips (text).

airline, the judges have the advantage. A deployed loop does not know in advance which situation it is in, so a judge that is beaten by a free rule in one domain and beats it in another needs a domain baseline before its accuracy means anything.

What the oracle’s knowledge does. Our corrections come from an oracle that sees grading criteria. This makes rescue achievable, but it also favours steps where knowing the expected outcome suffices to correct the action, such as writes with the right arguments. The judges see the same criteria, so the gap to the first-write rule is not an information gap. Counterfactual search, which corrects without them, usually had a rescuing step among its suspects but could not confirm it with its own corrections; an attributor that replays needs corrections as good as the oracle’s before replays pay for themselves.

Implications for self-improving loops. The analyst in RRSI-style loops is a trace digester; on our failures it was among the least accurate attributors. Yet replacing its analysis with a more accurate step did not give better repairs (§6). We see two possible reasons, neither tested here. The proposer reads the traces itself, grading report included, and may find the same mistakes whichever step it is pointed to; a loop that hid the traces would test attribution more directly. And a repair’s value depended less on where the failure lay than on how the edit

was built: the largest losses came from write gates and argument rewrites that misfired broadly, and RRSI’s re-evaluation on training tasks rejected each of them. Our results suggest that in a loop like this one, the evaluation that decides which repairs to keep protects the agent more than the accuracy of the attribution behind them.

8 Conclusion

Against a ground truth defined by counterfactual rescue with null controls, LLM attribution of agent failures did not beat a free structural rule on 300 natural failures in three domains, and replays and larger models did not close the gap. Giving a self-improvement loop’s proposer more accurate steps did not make its repairs better on held-out tasks. The earliest-step ground truth common in this literature is unstable once several steps can rescue a failure; a rescue profile is stable enough to compare methods. We release the failures, rescue profiles and code.

Limitations

All agents, oracles and attributors use one model family (DeepSeek-V4). The failures come from one self-improvement loop and three domains, with 37 to 65 rescuable failures each, so per-domain intervals are wide. Ground-truth corrections are written by an oracle that sees grading criteria, which may favour write actions (§7); a different oracle

would give a different profile, and the retest measures only the oracle’s sampling variation. The threshold of 0.5 for rescuability and the use of four samples per step were fixed on a 20-failure pilot. Our methods were prompted for the earliest decisive mistake while the primary metric rewards any rescuing step; agreement with the earliest step is reported as a secondary metric and orders the methods similarly. Fourteen retail failures were drawn after the others, from two further runs of the same loop, because the first did not yield enough distinct retail failures. Two of 6,540 AppWorld replays could not run (Appendix D). Who&When is a multi-agent benchmark with a different format; only methods without replay were run there. The repair experiment covers τ^2 only, with eight states and 16 candidate slots per group. For the first write and binary search it rules out differences from RRSI’s analysis larger than 2.5 points, not smaller ones, and its contrasts with no attribution are dominated by one candidate. Its digests carried only the step, not a reason or a component, and its proposers could read every trace with its grading report, so it tests whether pointing a proposer at a better step helps, not whether a better diagnosis would.

Ethics Statement

All environments use synthetic users and data. Attribution methods that misdiagnose failures can lead self-improving agents to change the wrong behaviour; our results suggest that such diagnoses should be checked against simple baselines before they drive automatic changes.

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.

Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

Joseph Y. Halpern. 2016. *Actual Causality*. MIT Press.

Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-Thu Le. 2026. Towards self-improving error diagnosis in multi-agent systems. *arXiv preprint arXiv:2604.17658*.

Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. 2026. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. *arXiv preprint arXiv:2606.09071*.

Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&when pro: Can LLMs really attribute failures in AI agents? *arXiv preprint arXiv:2607.09996*.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. *arXiv preprint arXiv:2512.06749*.

Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference*, 2nd edition. Cambridge University Press.

Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate failure prediction in agents does not imply effective failure prevention. *arXiv preprint arXiv:2602.03338*.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. *arXiv preprint arXiv:2609.24972*.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026a. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Ground-Truth Details

Replays. τ^2 replays restore the database state after the kept steps and continue with the same agent harness and user simulator (DeepSeek-V4-Flash, temperature 0); a corrected action replaces the agent’s turn at step k (a message, or tool calls whose results are then computed). AppWorld replays restore the app state by re-executing the kept code and force the corrected turn, which must contain one Python block; the episode continues with the incumbent harness. A correction the replay cannot apply counts as giving no gain.

Retest. The retest failures were drawn with a seeded random choice of 10% per domain before any ground truth was computed and run with fresh oracle samples and replays.

Cost. The ground truth cost \$11.1 for the first 186 τ^2 failures and \$13.7 for AppWorld (plus \$0.7 for its retest); all parts of the 14 τ^2 failures added later cost about \$1.5. The attribution study, including methods, search, pilots and Who&When, cost about \$58. The repair experiment (§6) cost about \$35: \$6.4 for drafting and critique, \$1.8 for attribution and search replays, \$7.9 for RRSI’s evaluations and smoke tests (5,639 episodes) and \$18.9 for held-out deployment (14,880 episodes).

B Prompts

The τ^2 prompts follow, verbatim; braces mark what is filled in. The AppWorld prompts replace the description of the agent and the task, the grading report and the component descriptions with AppWorld’s (the code agent, its unit tests, and the same nine components worded for it), and the oracle and search ask for one assistant turn with one Python block.

Shared introduction (all judges)

A customer-service agent (an LLM inside a harness) failed a task of tau2-bench. Below is the failure as the developers’ analysis tools see it: the customer’s hidden instructions, the conversation (each agent step is numbered), and the grading.

The DECISIVE MISTAKE is the earliest agent step such that, had the agent acted correctly at that step and continued normally, the task would most likely have succeeded.

Harness components (a fix to the harness changes one of these):

- prompt: the system prompt text (instructions and policy)
- wording the model reads)
- control_flow: the agent loop code: how the next action is produced, checks or gates applied to tool calls before they run, retries, enforced confirmations
- config: numeric settings such as temperature, token limits or step limits
- output_plumbing: how tool results and tool descriptions are rendered to the model
- context_mgmt: which parts of the conversation history the model sees
- client_tool: extra tools the harness gives the model (e.g. a checker or calculator)
- skill: instruction files loaded for specific situations
- memory: structured state the harness keeps across steps
- subagent: an extra model call, e.g. a second model reviewing an action

```
=== FAILURE ===
{the failure: hidden instructions, numbered steps, grading
  → report}
```

All-at-once

Find the decisive mistake and the harness component whose change would most likely have prevented it. Answer with JSON:

```
{
  "step": <step number>, "top3": [<the three most likely step numbers, best first>],
  "component": "<component>", "components3": ["<three most likely components, best first>"],
  "reason": "<one sentence>"}
```

754 **Step-by-step (asked for $k = 0, 1, \dots$ on the**
755 **episode up to step k)**

Judge step $\{k\}$ only, given what happened before it. Is the
→ agent's action at step $\{k\}$ the decisive mistake (the
→ earliest step whose correction would most likely have made
→ the task succeed)? Answer with JSON:
{ "decisive": true | false, "component": "<component, if
→ decisive>", "reason": "<one sentence>" }

756 **Binary search (episode up to step hi)**

The decisive mistake is assumed to lie between step $\{lo\}$ and
→ step $\{hi\}$. Is it in the first half (steps $\{lo\}$ to $\{mid\}$) or
→ in the second half (steps $\{mid + 1\}$ to $\{hi\}$)? Answer with
→ JSON:
{ "half": "first" | "second", "component": "<the component most
→ likely at fault>",
→ "reason": "<one sentence>" }

757 **Root-cause attributor**

An LLM agent failed a task. Find the root cause of the failure.

Agent components (id: role):
{COMP_TEXT}

Trace:
{view(rec)}

Give (1) a one-sentence root cause describing the cause, not
→ the symptom;
(2) the index of the earliest step where the agent went wrong;
(3) the single component whose change would most plausibly
→ prevent this failure.
Answer with JSON: { "root_cause": "...", "step": <int>,
→ "component": "<component id>" }

758 **Ground-truth oracle (system prompt)**

You construct ground truth for failure attribution. A
→ customer-service agent failed a task of tau2-bench. You see
→ the domain policy, the agent's tools, the task's grading
→ criteria (the database must end as if exactly the gold
→ actions were executed, and the agent must tell the user
→ every required piece of information), and the conversation
→ up to one step of the agent.

Decide whether the agent's action at that step is a mistake: an
→ action a careful agent following the policy would not take
→ on the way to the gold outcome. Actions that are correct,
→ or harmless and do not lead away from the gold outcome, are
→ ok.

If it is a mistake, give the single action a careful agent would
→ take at that step instead: one assistant turn, either a
→ message to the user or one or more tool calls. Correct only
→ this step; do not do more than a careful agent would do at
→ this point. The corrected action must be one the agent
→ could have taken with what it knew then (the conversation
→ and tool results so far, and the policy); use the grading
→ criteria only to know what the right outcome is, never to
→ supply ids, amounts or facts the agent had not yet learned.

Answer with JSON:
{ "verdict": "ok" | "mistake", "why": "<one sentence>",
→ "action": { "message": "<text to the user>" | { "tool_calls":
→ [{ "name": "<tool>", "arguments": { ... } }] } } }
("action" only when the verdict is mistake.)

759 **Counterfactual search, suspect call (system**
760 **prompt)**

A customer-service agent on tau2-bench failed its task: the
→ customer's request was not resolved as the domain policy
→ requires. You see the domain policy, the agent's tools and
→ the conversation; you do not know what the correct outcome
→ was.

Name the steps where the agent most likely went wrong, most
→ suspect first (at most 5). For each, give the single action
→ a careful agent would have taken at that step instead (one
→ assistant turn: a message to the user or one or more tool
→ calls, using only what the agent knew at that point) and
→ the harness component most likely at fault.

Harness components:
- prompt: the system prompt text (instructions and policy
→ wording the model reads)
- control_flow: the agent loop code: how the next action is
→ produced, checks or gates applied to tool calls before they
→ run, retries, enforced confirmations
- config: numeric settings such as temperature, token limits or
→ step limits
- output_plumbing: how tool results and tool descriptions are
→ rendered to the model
- context_mgmt: which parts of the conversation history the
→ model sees
- client_tool: extra tools the harness gives the model (e.g. a
→ checker or calculator)
- skill: instruction files loaded for specific situations
- memory: structured state the harness keeps across steps
- subagent: an extra model call, e.g. a second model reviewing
→ an action

Answer with JSON:
{ "suspects": [{ "step": <int>, "why": "<one sentence>",
→ "component": "<component>",
→ "action": { "message": "<text>" | { "tool_calls": [{ "name":
→ "<tool>", "arguments": { ... } }] } } }] }

Repair experiment: notes to the proposer.

RRSI's proposer prompt is unchanged. In the
no-attribution group the round's analysis report
is replaced by the first note below; the step
groups get the second note and one digest per
failing trace, { "task_id", "lens": "failure",
"decisive_step": k , "step_excerpt": <the
step, at most 600 characters>, "blocker":
"failure decided at step k " }.

No analysis report this round. Read the traces yourself
→ (list_traces, then read_trace) to find what went wrong; the
→ failing traces are listed first.

No analysis report this round. Read the traces yourself
→ (list_traces, then read_trace) to find what went wrong; the
→ failing traces are listed first. For each failing trace,
→ PER-TASK DIGESTS give decisive_step: the step at which an
→ automatic failure-attribution method located the failure.
→ Read the trace around that step (read_trace with
→ from_step/to_step) to see what went wrong there.

C Secondary Metrics and Search Budgets

D Registration and Deviations

The registration (methods, prompts, metrics, the
primary comparison and the budget) was commit-
ted before any method ran on the main set. Changes
to the attribution study, each logged before the data
it affected were analysed:

- **Ground truth.** The registration left the pro-
tocol to be chosen on the pilot by test-retest
agreement. After the earliest-step protocol
agreed on 42% of pilot pairs, protocol 3 and
the rescue-gain metrics replaced it (Adden-
dum 1), while methods were running and be-
fore any main-set ground truth existed.
- **Retail top-up.** The first loop's harnesses
yielded 86 distinct retail failures under the
sampling rule; 14 more were drawn with the

Method	Model	Exact [95%]	Within 1	Earliest	Share of max	$R(\hat{k})$, all failures
First write	rule	0.38 [0.27, 0.49]	0.62	0.33	0.52	0.23 [0.17, 0.30]
Last step	rule	0.13 [0.07, 0.20]	0.43	0.08	0.25	0.11 [0.07, 0.16]
All-at-once	Flash	0.28 [0.20, 0.36]	0.52	0.31	0.40	0.19 [0.14, 0.23]
All-at-once	Pro	0.31 [0.22, 0.39]	0.52	0.32	0.39	0.17 [0.13, 0.22]
All-at-once	Pro, thinking	0.32 [0.22, 0.42]	0.60	0.36	0.42	0.18 [0.14, 0.23]
Step-by-step	Pro	0.19 [0.13, 0.26]	0.38	0.22	0.25	0.11 [0.07, 0.15]
Binary search	Pro	0.32 [0.24, 0.40]	0.58	0.34	0.43	0.19 [0.15, 0.24]
Root-cause attributor	Pro	0.28 [0.19, 0.37]	0.49	0.31	0.36	0.16 [0.12, 0.20]
RRSI trace digester	Pro	0.21 [0.14, 0.29]	0.41	0.22	0.29	0.14 [0.10, 0.17]
Counterfactual search@40	Pro, replays	0.26 [0.18, 0.35]	0.44	0.31	0.35	0.15 [0.11, 0.19]

Table 4: Secondary metrics, pooled over the three domains. Exact, within one step, earliest rescuing step and share of the largest gain are over rescuable failures; the last column is over all 300 failures.

Budget	Retail	Airline	AppWorld	Pooled	\$
8	0.22	0.13	0.28	0.22	0.0125
16	0.21	0.23	0.27	0.23	0.0184
24	0.22	0.31	0.27	0.26	0.0212
32	0.22	0.31	0.28	0.26	0.0222
40	0.22	0.31	0.28	0.26	0.0222

Table 5: Counterfactual search: mean $R(\hat{k})$ on rescuable failures by replay budget, read off the same runs, and dollars per failure.

two slots and the permutation split the copies between groups, which is not a valid test. We noticed this after the first run of the analysis, which gave $p = .009$ for first write against no attribution, and replaced the test by an exact sign-flip test over the eight state differences ($p \geq .25$); estimates and intervals are unchanged.

same rule from two further runs of the loop, as registered. All methods moved by at most 0.02 when they were added.

- **AppWorld.** Registered in Addendum 2 before any AppWorld failure was drawn. Two corrected replays of one step failed on every attempt because the oracle’s code passed a literal ellipsis that AppWorld’s request log cannot serialise; they count as corrections not applied. That failure’s other corrections at the step rescued nothing and its largest gain was 0.375, so they could not have made it rescuable.

Repair experiment. Registered separately, before any attribution was run on its traces and before any candidate was drafted. The first state served as a pilot; it exposed no bug, so its cells were kept, as registered. Two operational problems had no effect on data: a lookup by abbreviated commit hash re-invoked finished deployments, which skipped every episode as already done, and a container restart interrupted the deployments, which resumed with finished episodes kept. One secondary test changed. The registered code tested the round-level contrasts by permuting slots, but a round has one accepted candidate, so each cell’s value was copied to its

Method	Model	Hand-crafted (58)		Generated (126)	
		agent	step	agent	step
All-at-once	Flash	0.59	0.09	0.58	0.21
All-at-once	Pro	0.62	0.03	0.67	0.18
All-at-once	Pro, thinking	0.62	0.15	0.65	0.25
Step-by-step	Pro	0.24	0.19	0.44	0.32
Binary search	Pro	0.26	0.09	0.38	0.04

Table 6: Who&When, with ground truth shown to the judge as in its authors’ setting: accuracy of the responsible agent and step.